CS6008 - Network Security

Assignment: Building a Secure Chat Application

Course: Network Security / CS6008
Assignment Weight: 10%
Duration: 14 days
Team Size: Individual or pair of two.

1. Overview

In this assignment, you will design and build a one-to-one chat application over TCP sockets, and progressively harden it against a series of realistic network attacks. Rather than implementing security "all at once," you will build the system in five phases, each of which adds exactly one security property. After each phase that introduces a cryptographic protection, you are required to attempt to break your own previous phase using a tool you build yourself, and show **with evidence** whether the new phase actually stops that attack.

This mirrors how real-world secure protocols are motivated and evaluated: a security mechanism is only meaningful if you can demonstrate the specific threat it defeats, not merely assert that it is "secure."

1.1 Learning Objectives

- Implement a working client–server chat protocol using TCP sockets.
- Implement the Diffie–Hellman key exchange from first principles (using modular exponentiation), and understand why the raw shared secret must be hashed before being used as a symmetric key.
- Use authenticated encryption (AES-GCM) correctly, and understand why it differs from plain AES.
- Understand why key exchange alone does not provide authentication, by building a working man-in-the-middle (MITM) attack against your own unauthenticated protocol.
- Set up a minimal Public Key Infrastructure (PKI) with OpenSSL: a Certificate Authority, a certificate signing request, and certificate issuance and validation.
- Understand the difference between a certificate proving an identity was vouched for, and a live connection proving possession of the matching private key.
- Understand and implement end-to-end encryption between clients in the presence of an untrusted (or semi-trusted) relay server.
- Understand and implement forward secrecy via periodic key rotation, and articulate what specific guarantee it adds beyond a single long-lived session key.

1.2 Prerequisites / Allowed Tools

- **Use C/C++ with POSIX complaint functions.**
- For verifying the generated certificate during Phase 3, use **OpenSSL's s_client**.
- Wireshark, for traffic verification at multiple phases.
- Standard cryptographic libraries may be used in C/C++ for AES-GCM encryption, SHA-256 hashing, and X.509 certificate parsing (e.g., through OpenSSL). However, the Diffie–Hellman key exchange must be implemented from scratch, including the required modular exponentiation. You may not use any library-provided DH or ECDH implementation/class for Phases 2–5.
- **You should not use any TLS/SSL library or wrapper (e.g. OpenSSL's `<openssl/ssl.h>`) or any built-in Diffie–Hellman API (e.g. `<openssl/dh.h>`) anywhere in this assignment — you must implement the key**

exchange, encryption, and certificate validation logic yourself. However, low-level cryptographic primitives (e.g. `<openssl/bn.h>`, `<openssl/evp.h>`, `<openssl/x509.h>`) can be used as building blocks.

1.2.1 Deployment Environment — Virtual Machines (Required)

This assignment must be implemented and demonstrated across separate virtual machines, not solely on a single machine via loopback. This mirrors a real network deployment and makes your Wireshark captures and MITM attack demonstrations meaningful rather than artifacts of same-machine testing.

- Minimum topology: three VMs — one Server VM (S) and two Client VMs (C1, C2), each on the same virtual network (e.g., VirtualBox/VMware), each with Ubuntu OS or any Linux distro.
- For the attack tasks in Phase 2 and Phase 3 (§3.3, §4.2), use a fourth VM ("Mallory") positioned on the same network to run your MITM proxy, distinct from both the Server VM and the Client VMs.
- Networking setup: Verify basic connectivity between VMs with ping before running your chat application.
- Certificate CN (Phase 3 onward): CA can be hosted on the Server VM. Clients must validate against this same value.
- Include, in your report, a brief note or diagram of your VM topology (IP addresses and roles) alongside each phase's screenshots, so the TA can correlate captures with your network layout.

1.3 Required Client Command Interface

Your client must support a simple, tag-based command interface, typed directly at the chat prompt. Any input that does not match one of the recognized command tags below should be treated as a plain chat message to whichever user is currently selected. The following commands are required from the phase noted; you may add further commands of your own, but these must work exactly as specified so your application (and its behaviour) can be evaluated consistently.

Command	Required from	Behaviour
`@username message`	Phase 1	Sends a message to username, and sets username as the currently selected chat partner.
`/chat username`	Phase 1	Switches the currently selected chat partner to username, without sending a message.
`/who`	Phase 1	Requests and displays the list of currently online users from the server.
`/quit`	Phase 1	Cleanly disconnects from the server and exits the client.
`/e2e username`	Phase 4	Initiates an end-to-end key exchange between clients with username, independent of the server (see §5).

Design note: Your chat application should follow the same command tags as mentioned above. **Do not rename** them, since graders and any provided test scripts will use these exact strings. If your design in Phase 5 needs additional user-facing commands (e.g. to check a key's current age, or to force an immediate rekey for testing), you are free to add them, but must document them clearly in your Report.

1.4 Wire-Level Tagging Convention (Phases 4–5) between the clients

From Phase 4 onward, your protocol needs a way for the receiving side to tell apart a handshake message from an actual chat message, since both travel over the same @username-addressed channel through the server. You must use the following tag convention:

Wire Tag	Meaning
`__E2E_INIT__<data>`	**End-to-End Session Initialization:** Indicates the initiation (or re-initiation) of an end-to-end key exchange. The `<data>` field contains the cryptographic parameters or key-exchange information required by the peer.
`__E2E_ACK__<data>`	**End-to-End Session Acknowledgment:** Indicates a response to an `__E2E_INIT__` message and carries the corresponding key-exchange data required to complete the end-to-end key establishment.
`__E2E_MSG__<data>`	**End-to-End Encrypted Message:** Indicates application-layer chat data encrypted using the established end-to-end session key. The `<data>` field contains the resulting encrypted payload.

You convention must satisfy two properties: (1) the server's existing @username routing logic must not need to change or understand these tags, it should keep relaying opaque text exactly as it did in Phase 2/3; and (2) a message tagged as a handshake step must never be shown to the user as chat content, and a message tagged as chat content must never be silently treated as a handshake step.

1.5 Submission Requirements

- **Source Code:** Submit all source code organized into phase-specific directories (e.g., phase1/, phase2/, etc.). Each directory must be independently executable and include a brief README with execution instructions.
- **Git commits:** Initialize git in the local assignment folder. The repo should have incremental commits after each phase or at each checkpoint, with a proper commit message within each phase. The TA will verify.
- **Written Report:** Submit a single PDF report documenting all five phases. For each phase, describe your implementation and verification steps. For Phases 2–5, include the results of required attack/verification tasks, supported by relevant logs or screenshots.
- **Generative AI usage:** You may utilize LLMs to assist with this assignment. However, you must include an appendix in your report detailing all prompts used.

2. Phase 1 - Baseline Chat Application (No Security)

2.1 Requirements

- Implement a chat server that supports exactly one server (S) and two clients (C1, C2) connected simultaneously over TCP.
- Any message typed by C1 must be delivered to C2 via the server, and vice versa.
- The server relays each message as-is (no encryption).
- Implement the client command interface specified in §1.3 (@username, /chat, /who, /quit) - this applies starting from this phase and must continue to work in every later phase.

2.2 Required Verification

- Modify or instrument your server so it prints/logs every message it relays, and confirm in your report that the server itself can read the full content of every message.

- Capture live traffic between a client and the server using Wireshark (filter on the relevant VM's network interface — e.g., eth0 — and your chat port; see §1.2.1), and use "Follow -> TCP Stream" to show the message content is visible in plaintext in the raw capture. Include a screenshot in your report.

2.3 Deliverables for This Phase

- Server and client source code.
- A short section in your report describing your protocol (message framing, how you decided a message is "complete," how the server routes messages if more than two clients could theoretically connect).
- The Wireshark screenshot described above.

3. Phase 2 - Client-Server Confidentiality via Diffie–Hellman

3.1 Requirements

- Implement the Diffie-Hellman key exchange yourself, using modular exponentiation. You must use a standard, published prime group (e.g., an RFC 3526 MODP group) rather than generating your own prime.
- Each client must perform an independent DH key exchange with the server when it connects: one exchange for C1 <-> S, and a separate, independent exchange for C2 <-> S.
- Derive a symmetric AES key from the raw DH shared secret using a cryptographic hash function (do NOT use the raw shared secret directly as a key). Why should we hash the key? Mention this in your report.
- Encrypt every subsequent message on each link with an authenticated encryption mode (AES-GCM is recommended). This should include not just chat messages but also any login/registration exchange your application performs, if applicable.

3.2 Required Verification

- On both ends of a given DH exchange, print a hash-based fingerprint of the derived shared secret (never print the raw secret or the key itself). Show with evidence that both sides computed the identical fingerprint.
- Repeat the Wireshark capture from Phase 1 on the same traffic. Confirm the message content is no longer readable, and include a screenshot showing the ciphertext where Phase 1 showed plaintext.
- Demonstrate that your AES-GCM usage detects tampering: modify a single byte of a captured ciphertext and attempt to have your client process it. Show that this is rejected (e.g., an authentication/decryption failure) rather than silently producing corrupted output.

3.3 Attack Task - Man-in-the-Middle

Diffie-Hellman, by itself, establishes a shared secret with whoever answers the connection; it does not authenticate who that is. Your task:

- Build a separate proxy program that sits between a client and your Phase 2 server, and performs two independent DH exchanges: one with the victim client (posing as the server), and one with the real server (posing as the client).
- Show that your proxy can read and log every message exchanged, even though both the client and the server believe they are communicating directly and securely with each other.
- In your report, explain what observable evidence (if any) would have allowed the victim to detect that this attack was happening, and why an ordinary user would be unlikely to notice it.
- Run your MITM proxy on the separate Mallory VM described in §1.2.1, positioned on the network between a Client VM and the Server VM. The victim client is explicitly configured to connect to Mallory's proxy address/port instead of the real server's address (i.e., you manually point the client at Mallory). This is sufficient to satisfy the core requirement of demonstrating two independent DH exchanges and captured plaintext.

3.4 Deliverables for This Phase

- Your DH implementation, client, and server source code.
- Your MITM proxy source code.
- Fingerprint-matching evidence, Wireshark screenshots (before/after), and your tamper-detection test.
- A short section demonstrating the MITM attack succeeding, with logs/screenshots showing captured plaintext.

4. Phase 3 - Server Authentication via PKI

4.1 Requirements

- Using OpenSSL, create your own Certificate Authority: a private key and a self-signed root certificate.
- Using OpenSSL, generate a key pair for your chat server, create a Certificate Signing Request, and have your CA sign it to produce a server certificate.
- Modify your Phase 2 protocol so that, before any DH exchange, the server sends its certificate to the connecting client.
- The client must validate the received certificate against its own trusted copy of the CA's root certificate before proceeding. At minimum, validate: (a) the certificate's signature was produced by the trusted CA, (b) the certificate's validity period, and (c) the certificate identifies the expected server. If validation fails for any reason, the client must abort the connection immediately; it must not send a password, proceed with DH, or reveal any further information.
- Additionally, the server must prove it possesses the private key corresponding to its certificate (not merely a copy of the certificate file). Design and implement a mechanism for this. For example, having the server sign some value from the handshake with its private key, which the client verifies using the public key embedded in the certificate.

4.2 Required Verification

- Demonstrate the full legitimate flow: client connects, receives and validates the certificate, verifies proof-of-possession, and completes the DH exchange as in Phase 2.
- Re-run your Phase 2 MITM proxy against this new protocol. Since your proxy does not have access to your CA's private key, it cannot produce a validly-signed certificate. Show, with logs/screenshots, that the client detects this and refuses to proceed. Contrast this explicitly against the Phase 2 result, where the same attack succeeded.
- Perform this re-attempt using the same Mallory VM setup from Phase 2 (§3.3) — including, if you implemented the recommended fully transparent variant, the ARP-spoofed/redirected configuration — to confirm the certificate validation defense holds under the same network conditions where the Phase 2 attack succeeded.
- Separately, demonstrate what happens if an attacker obtains a copy of your real server certificate file but not the corresponding private key and attempts to use it. Your proof-of-possession mechanism should detect and reject this case too, and show evidence.

4.3 Deliverables for This Phase

- Your OpenSSL commands/scripts used to set up the CA and issue the server certificate.
- Updated client/server source code implementing certificate exchange, validation, and proof-of-possession.
- An updated version of your MITM proxy is attempting the attack against this phase, plus evidence that it now fails.
- The proof-of-possession bypass attempt and evidence that it was rejected.

5. Phase 4 - End-to-End Encryption Between Clients

5.1 Requirements

- Design and implement a mechanism so that C1 and C2 can establish a shared secret directly with each other. independent of, and never derivable by, the server, while still using the server purely as a message relay.

- This exchange must be triggered by the /e2e username command specified in §1.3, and must use a wire-level tagging convention along the lines described in §1.4 so the server's routing logic does not need to change.
- The server MAY continue to see routing metadata (i.e., which username is sending to which username), since it needs this to deliver messages. The server must NOT be able to derive the C1 <-> C2 shared key or decrypt the resulting message content.
- Chat messages between C1 and C2, once this session is established, must be encrypted with the C1 <-> C2 key before being sent over the existing (already-encrypted) client-server link from Phase 3. In other words, this is an additional, inner layer of encryption, not a replacement for the Phase 2/3 links.
- Your design should clearly distinguish, at the protocol level, between handshake messages (establishing the C1 <-> C2 key) and encrypted chat messages, so the server's existing relay logic does not need to understand cryptography - it should just be forwarding opaque data by username, exactly as it did in earlier phases.

5.2 Required Verification

- Show, with independently-computed fingerprints on both C1 and C2, that they derive the identical shared secret without the server's involvement.
- **Instrument your server to log what it relays. After the C1 <-> C2 session is established, show that the server's log contains only unreadable/opaque data for chat content. Include a screenshot or log excerpt, and contrast it against a message sent before the session was established (which the server could read, matching Phase 2/3 behaviour).**
- Confirm end-to-end functional correctness: a message sent by C1 must be correctly decrypted and displayed by C2.

5.3 Deliverables for This Phase

- Source code implementing the client-to-client key exchange and message wrapping.
- Fingerprint-matching evidence and server-log evidence as described above.

6. Phase 5 - Forward Secrecy

6.1 Requirements

- Extend your Phase 4 end-to-end session so that the shared key between C1 and C2 is automatically renegotiated on a fixed timer (every 60 seconds) for as long as the session remains active.
- Each renegotiation must produce a genuinely new key, independent of the previous one, and the old key must be discarded and never reused once the new key is confirmed.
- Your design must handle the case where both clients might attempt to trigger a renegotiation at (approximately) the same time, without the two sides ending up with different, disagreeing keys as a result. Explain and justify your chosen approach in your report.
- Ongoing chat should not be disrupted by a key rotation. Messages sent shortly before, during, and after a rotation should all still be delivered and readable.

6.2 Required Verification

- Log, on both clients, a fingerprint of the current key and a timestamp whenever a rotation occurs. Show, across at least two rotations, that (a) the fingerprint changes each time, (b) both clients always agree on the current fingerprint after a rotation completes.
- Demonstrate that a chat message sent immediately after a rotation is correctly decrypted by the recipient (i.e., the rotation did not break the live session).
- In your report, explain in your own words, and specifically in terms of what an attacker who later compromises a single key can and cannot do - what forward secrecy actually buys you that Phase 4 alone did not.

6.3 Deliverables for This Phase

- Source code implementing the rekey timer and collision-avoidance logic.

- Rotation evidence (fingerprints + timestamps, both sides) and the post-rotation message delivery test.
- The written explanation described above.

7. Evaluation Rubric

Component	Weight
Phase 1 - baseline chat + plaintext verification	10%
Phase 2 - DH implementation, AES-GCM encryption, verification	20%
Phase 2 - working MITM attack + explanation	10%
Phase 3 - CA/PKI setup, certificate validation, proof of possession	20%
Phase 3 - MITM re-attempt showing the attack is now defeated	10%
Phase 4 - end-to-end encryption + server-blindness verification	15%
Phase 5 - forward secrecy + rotation verification	10%
Report quality (clarity, evidence, correct explanations)	5%